

Assignment 2

Aim: Implement merge sort and quick sort and calculate space and time complexity.
(Compute order of complexity for each function and then calculate overall complexity)

```
#include <stdio.h>
```

```
void merge (int arr[], int left, int mid, int right);
void mergeSort (int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort (arr, left, mid);
        mergeSort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}
```

```
void merge (int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i]; i++;
        } else {

```

```

        arr[k] = R[j]; j++;
    }
    k++;
}
while (i < n1) {
    arr[k] = L[i] L[i];
    i++; k++;
}
while (j < n2) {
    arr[k] = R[j];
    j++; k++;
}
}

```

```

void swap (int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```

int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++; swap (&arr[i], &arr[j]);
        }
    }
    swap (&arr[i+1], &arr[high]);
    return (i+1);
}

```

```
void quickSort(int int arr[], int low, int high) {
    if (low < high) {
        int partitionIndex = partition(arr, low, high);

        quickSort(arr, low, partitionIndex - 1);
        quickSort(arr, partitionIndex + 1, high);
    }
}
```

```
void main() {
    int arr1[] = {10, 27, 38, 3, 9, 2, 42};
    int arr2[] = {10, 27, 38, 3, 9, 2, 42};
    int size = 7;
    printf("Original array:\n");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr1[i]);
    }

    printf("\n");
    mergeSort(arr1, 0, size - 1);
    quickSort(arr2, 0, size - 1);
    printf("Array sorted using merge sort: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr1[i]);
    }

    printf("\n Array sorted using quick sort: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr2[i]);
    }
}
```

Function	Time Complexity	Space Complexity
mergeSort()	$O(n \log n)$	$O(\log n)$ (recursion)
merge()	$O(n)$	$O(n)$
MergeSort overall	$O(n \log n)$	$O(n)$
quickSort()	$O(n \log n)$	$O(\log n)$
Best / Average		
quickSort()	$O(n^2)$	$O(n)$
Worst		
partition()	$O(n)$	$O(1)$
swap()	$O(1)$	$O(1)$